Controlled Empirical Analysis of Single-Task and Multi-Task Learning for Semantic Segmentation and Drivable Area Detection

Nishit Vinod Kumble
MSC.Artificial Intelligence
Lawrence Technological University
Southfield, Michigan
nkumble@ltu.edu

***Abstract*—This project presents a comparative study between Single-Task Learning (STL) and Multi-Task Learning (MTL) for semantic segmentation and drivable area detection in autonomous driving scenes. The experiments were conducted using the BDD100K, specifically the 10K subset, which provides RGB images, semantic segmentation masks, and drivable area labels. Three models were implemented: an STL semantic segmentation model, an STL drivable area model, and an MTL model with a shared ResNet-18 encoder and separate decoder heads for both tasks. All models were trained using Adam optimizer with early stopping and evaluated using validation loss, pixel accuracy, and mean Intersection over Union (mIoU). The experimental results showed that the MTL model slightly improved semantic segmentation performance compared to the STL baseline, achieving a segmentation mIoU of 0.2945 compared to 0.2814. However, drivable area detection performance decreased under MTL; with the STL model achieving a higher mIoU of 0.7003 compared to 0.6508 for MTL. These findings suggest that shared representations can provide positive transfer for more complex tasks such as semantic segmentation, while simpler tasks such as drivable area detection may experience negative transfer due to reduced task-specific specialization.**

***Index Terms*—Autonomous Driving, Multi-Task Learning, Single-Task Learning, Semantic Segmentation, Drivable Area Detection, ResNet-18, Mean Intersection over Union.**

I. Introduction

Autonomous driving systems rely on multiple perception and prediction tasks to understand road scenes and make decisions The most important perception tasks are semantic segmentation and drivable area detection. A segmentation mechanism annotates the boundaries of vari ous types of objects and assigns different colors to each pixel identified to belong to different objects [1]. Drivable area detection can help intelligent vehicles understand road conditions and determine safe driving area [2]. These tasks are important for path planning, obstacle avoidance, and scene understanding.

A. Single-Task Learning

Traditionally, problems like segmentation and detection are usually dealt with using two independent models. Although this approach can achieve good performance, it increases computational cost, memory usage, and inference time because multiple models must be maintained. In autonomous driving systems, where real-time processing is important, using separate models for every perception task may not be efficient [3].

B. Multi-Task Learning

Multi-task learning (MTL) is one of the most well-known techniques for simultaneously dealing with multiple tasks, which can exploit the shared information of multiple related tasks and improve the performance of each other [3].In computer vision, related tasks often benefit from shared visual features such as road boundaries, lane structures, vehicles, and scene context. Because of this shared learning, MTL can reduce computational cost and may improve performance. However,in computer vision MTL does not always improve all tasks equally. Many seemingly related tasks exhibit negative transfer when trained jointly [4].

C. Motivation and Objective

Autonomous driving systems must perform several perception tasks simultaneously in real time. Training separate models for each task can increase memory usage, computational cost, and inference time. Multi-Task Learning offers a more efficient solution by allowing multiple tasks to share a common encoder while using separate task-specific heads. However, shared learning does not always improve all tasks equally. In some cases, one task may benefit from shared features while another may experience negative transfer due to competition for the same encoder.

Therefore, it is important to study whether Multi-Task Learning improves both semantic segmentation and drivable area detection simultaneously, or whether trade-offs emerge between the tasks. In this project, separate STL models and an MTL model were developed using the BDD100K dataset [5]. The models were compared using validation loss, pixel accuracy, and mean Intersection over Union (mIoU) to analyze positive and negative transfer effects between the tasks.

II. Related Work

This section surveys the body of work relevant to our empirical analysis, organized into four areas: (1) Foundational architectures for semantic segmentation, (2) Drivable area detection methods, (3) Multi-task learning frameworks for autonomous driving perception, and (4) Optimization challenges specific to joint training paradigms.

A. Semantic Segmentation Architectures

Semantic segmentation is the task of assigning a class label to every pixel in an image. It has undergone substantial architectural evolution over the past decade. The field was catalyzed by the introduction of Fully Convolutional Networks (FCNs) [6], which replaced the fully connected layers of classification CNNs with convolutional layers, enabling dense per-pixel prediction of arbitrary input sizes. Subsequent work revealed that aggressive max-pooling in FCN-style networks led to significant loss of spatial resolution. SegNet [7] addressed this by storing max-pooling indices in the encoder and reusing them during upsampling in the decoder, producing sharper segmentation boundaries while keeping parameter counts low.

The DeepLab family of models [8], developed by Google, introduced atrous (dilated) convolution to enlarge the receptive field without reducing spatial resolution, paired with Conditional Random Fields (CRF) for boundary refinement. DeepLabV3+ [9] added an encoder-decoder structure to recover fine-grained spatial details. More recently, hybrid approaches combining CNN encoders with lightweight MLP decoders such as SegFormer [10] have demonstrated strong performance with reduced computational overhead, making them attractive candidates for real-time deployment in autonomous vehicles.

B. Drivable Area Detection

Drivable area detection is identifying the region of the road where a vehicle may safely operate. It is a core perception task for autonomous driving systems. Early deep learning approaches treated it as a standard binary segmentation problem, applying FCN or SegNet-based pipelines to predict free-space masks [11].

Qiao and Zulkernine [2] systematically evaluated deep learning-based semantic segmentation architectures for drivable area prediction on the BDD100K dataset, framing the problem as pixel-wise classification. Their work demonstrated that encoder-decoder models with multi-scale feature extraction could achieve over 95 percent mAP at real-time inference speeds, establishing competitive single-task baselines against which multi-task approaches can be compared.

The BDD100K dataset [5] has become the de facto benchmark for drivable area segmentation in diverse driving conditions (day/night, weather variations). Its large-scale annotations for drivable regions, lane lines, and object detection have catalyzed unified multi-task evaluation, enabling direct comparison between single-task and multi-task approaches under controlled conditions, a practice central to our empirical study.

C. Multi-Task Learning Frameworks for Autonomous Driving

Multi-task learning (MTL) trains a single model with shared representations across multiple objectives, offering advantages in computational efficiency, data augmentation effects, and potentially improved generalization [12] . In autonomous driving, the dominant MTL paradigm couples an encoder backbone with multiple task-specific decoder heads. MultiNet [13] was among the earliest unified networks handling classification, detection, and road segmentation, using a shared ResNet encoder with three separate decoder heads.

The YOLOP network [14] marked a significant milestone as the first architecture to achieve real-time performance on all three core perception tasks namely, traffic object detection, drivable area segmentation, and lane detection simultaneously on an embedded device (Jetson TX2 at 23 FPS). Built atop a CSPDarknet encoder, YOLOP demonstrated through ablative experiments that joint end-to-end training outperforms alternating optimization strategies and that the three tasks benefit from shared feature representations. This established an important empirical baseline: tasks involving spatial grid-based prediction share complementary inductive biases. Building on YOLOP, HybridNets [15] (Vu et al., 2022) adopted EfficientNet-B3 as the backbone with a BiFPN neck to improve multi-scale feature fusion. HybridNets achieved mAP of 77.3 percent on vehicle detection and lane accuracy of 85.4 percent on BDD100K, outperforming YOLOP on lane detection while being up to 1.4× faster. Notably, HybridNets merged the two segmentation heads of YOLOP into a single head, demonstrating that task-sharing can be pushed further without sacrificing performance. More recently, YOLOPv3 [16] (Zhan et al., 2024) addressed underutilized high-resolution features and non-local contextual dependencies by introducing transformer-augmented feature aggregation, achieving further gains on BDD100K. Similarly, the YOLO-ODL model [17] incorporated weight-balancing strategies to automatically adjust task loss contributions during joint training, reporting state-of-the-art accuracy-efficiency tradeoffs at 94 FPS on an RTX 3090. These works collectively demonstrate that architectural choices in the encoder-decoder design and the strategy for combining task-specific decoders critically determine whether multi-task networks match or exceed single-task baselines.

D. Optimization Challenges in Multi-Task Learning

A fundamental challenge in multi-task learning is ensuring that jointly training multiple objectives does not degrade performance relative to training each task independently. Caruana [12] identified this risk early, noting that while shared representations can provide beneficial inductive bias, tasks with misaligned objectives may interfere with one another in the shared parameter space, a phenomenon commonly referred to as negative transfer.

Kang [4] conducted an empirical study specifically examining negative transfer in MTL settings, analyzing the conditions under which joint training leads to performance degradation and surveying mitigation strategies. Their findings highlight that the degree of task relatedness is a critical factor: tasks

that share complementary features benefit from joint training, whereas tasks with diverging gradient signals can actively harm one another during optimization.

In the autonomous driving context, Zhao et al. [3] demonstrated that multi-task learning for dangerous object detection requires careful balancing of task-specific losses, as the detection and segmentation objectives operate at different scales and semantic granularities. Similarly, Wu et al. [14] investigated multiple training paradigms for YOLOP including end-to-end joint training and various alternating optimization schedules—and found that straightforward end-to-end training was sufficient when tasks were sufficiently related, but that the ordering and weighting of tasks in step-by-step training materially affected final accuracy on each head. More recently, Zhan et al. [16] addressed the problem of underexploited feature sharing across task decoders, proposing architectural modifications to reduce gradient conflicts arising from simultaneously optimizing detection and segmentation objectives.

These findings collectively motivate the controlled experimental design of the present study: by holding architecture, dataset, and training hyperparameters constant and varying only whether tasks are trained jointly or independently, we aim to isolate the true effect of multi-task coupling on segmentation and drivable area detection performance.

III. Methodology

This section describes the experimental design of the controlled empirical study, covering dataset preparation, model architectures, and training configuration. To ensure a fair comparison between single-task learning (STL) and multi-task learning (MTL), all models share an identical backbone, decoder design, optimizer, and training hyperparameters. The only variable that differs between conditions is whether the backbone is trained independently per task or jointly across both tasks.

A. Dataset Preparation

Experiments are conducted on a curated subset of the BDD100K dataset [5], a large-scale driving benchmark providing diverse urban driving imagery with annotations for multiple perception tasks. The full 10,000-image subset was pre-processed to retain only samples for which both a valid semantic segmentation mask and a valid drivable area mask were simultaneously available. Of the 7,000 images with usable segmentation labels, a significant portion lacked corresponding drivable area annotations. After filtering, 2,976 samples satisfied all three requirements — a valid RGB image, a clean segmentation mask, and a clean drivable area mask. It constituted the final dataset used in all experiments.

This intersection-based filtering strategy is a deliberate design choice: by training and evaluating all three model variants (STL-Seg, STL-Drive, MTL) on exactly the same 2,976 samples, any observed performance difference is attributable solely to the single-task versus multi-task training paradigm, and not to differences in training data. The dataset was split 80/20 into a training set of 2,380 images and a validation set of 596 images, with no overlap between splits. All images were resized to 512 × 256 pixels (width × height) prior to training.

TABLE I
Dataset Statistics and Configuration

Property	Value	Notes
Source	BDD100K (10k subset)	Yu et al. [5]
Raw images	7,000	After cleaning
Final samples	2,976	All three labels valid
Train / Val split	2,380 / 596	80% / 20%
Input resolution	512 × 256	W × H, pixels
Seg. classes	19	ignore_index = 255
Drivable classes	3	BG / Alt. / Direct

The semantic segmentation task covers 19 valid semantic classes drawn from the BDD100K label taxonomy, with pixel values of 255 used as the ignore label for unlabelled or ambiguous regions. The drivable area task is formulated as a three-class segmentation problem: background (non-drivable), alternative drivable area, and directly drivable area, consistent with the original BDD100K drivable annotation scheme [5].

B. Model Architecture

Three model variants are implemented and evaluated: a single-task segmentation model (STL-Seg), a single-task drivable area model (STL-Drive), and a joint multi-task model (MTL). All three variants share an identical encoder architecture to ensure architectural parity.

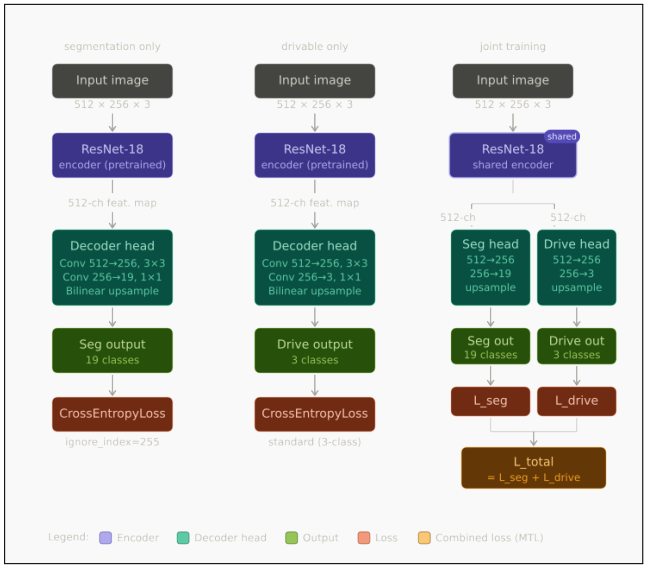

Fig. 1. Comparison of STL and MTL architectures.

- Encoder: A ResNet-18 backbone pre-trained on ImageNet is used as the shared encoder for all model variants. ResNet-18 produces a feature map of 512 channels at 1/32 of the input spatial resolution after its final convolutional stage. The use of a pre-trained backbone is consistent across all conditions, ensuring that any performance difference is not attributable to differences in initialisation.

TABLE II
MODEL ARCHITECTURE COMPARISON

Component	STL-Seg	STL-Drive	MTL
Encoder	ResNet-18	ResNet-18	ResNet-18 (shared)
Pretrained	ImageNet	ImageNet	ImageNet
Encoder output	512-ch feat. map	512-ch feat. map	512-ch feat. map
Decoder head(s)	1 (seg)	1 (drive)	2 (seg + drive)
Dec. Conv 1	512→256, 3×3	512→256, 3×3	512→256, 3×3 each
Dec. Conv 2	256→19, 1×1	256→3, 1×1	256→19 / 256→3
Upsample	Bilinear to 512×256	Bilinear to 512×256	Bilinear to 512×256
Output classes	19	3	19 + 3

- Decoder Heads: Each task is served by a lightweight FCN-style decoder head. The decoder consists of two convolutional layers followed by bilinear upsampling to the original input resolution. Specifically, the first convolutional layer reduces the 512-channel encoder output to 256 channels using a 3×3 convolution with padding 1, followed by a ReLU activation. The second convolutional layer then projects from 256 channels to the number of task-specific output classes using a 1×1 convolution. The resulting feature map is then bilinearly upsampled to the target resolution of 512×256. In STL-Seg, the output has 19 channels (one per semantic class). In STL-Drive, the output has 3 channels. In MTL, two independent decoder heads of this design are attached to the shared encoder — one producing 19-class segmentation logits and one producing 3-class drivable logits with no parameter sharing between the decoder heads themselves.
- The MTL architecture follows the hard parameter sharing paradigm [12], in which the encoder parameters are fully shared across tasks during joint training while the decoder parameters remain task-specific. This is the most widely adopted MTL architecture in the autonomous driving perception literature [14], [15] and provides the cleanest controlled comparison against the STL baselines.

C. Training Configuration

All models are trained for a maximum of 10 epochs using the Adam optimizer with a fixed learning rate of 1×10^{-4} and a batch size of 8. Early stopping is applied to all three models with a patience of 3 epochs, monitoring the validation loss after each epoch. Training is halted when the monitored metrics: avgvalloss for both STL models and valtotalloss for MTL, fails to improve for 3 consecutive epochs. No learning rate scheduling or weight decay was applied, to maintain identical conditions across all three models. In practice, all three models independently triggered early stopping at epoch 5, with each model achieving its best validation loss at epoch 2 and showing no further improvement over the subsequent three epochs (epochs 3, 4, and 5). This consistent convergence behaviour across all three independently trained models suggests that 5 epochs of effective learning is sufficient for the given dataset size and architecture, and that the results reported are not sensitive to the choice of maximum epoch budget.

- Loss Functions: Cross-entropy loss is used for both tasks across all model variants. For the segmentation task, CrossEntropyLoss is applied with ignore index=255 to exclude unlabelled pixels from the gradient computation. For the drivable area task, standard CrossEntropyLoss is applied over the three output classes. No class-frequency weighting is applied to either loss, ensuring that any class imbalance effects are held constant across conditions.
- MTL Loss Combination: In the MTL model, the total training loss is defined as the unweighted sum of the two task losses:

$$L_{\text{total}} = L_{\text{seg}} + L_{\text{drive}} \tag{1}$$

This equal weighting scheme assigns identical importance to both tasks throughout training, avoiding any implicit bias toward one task that would arise from manual loss scaling. While more sophisticated loss weighting strategies exist — such as uncertainty-based weighting or gradient normalisation, equal weighting is employed here deliberately, as the goal is to evaluate the effect of joint training under the simplest and most transparent conditions, rather than to optimise MTL performance per se.

TABLE III
TRAINING HYPERPARAMETERS

Hyperparameter	STL (both)	MTL
Optimizer	Adam	Adam
Learning rate	1×10^{-4}	1×10^{-4}
Batch size	8	8
Max epochs	10	10
Early stopping	Patience = 3 (val loss)	Patience = 3 (val loss)
Actual stop epoch	Epoch 5 (all STL)	Epoch 5
Loss (seg)	CrossEntropyLoss	CrossEntropyLoss
Loss (drive)	CrossEntropyLoss	CrossEntropyLoss
MTL loss weighting	N/A	Equal sum (1:1)
Ignore index	255 (seg)	255 (seg head)
Class weighting	None	None

D. Evaluation Metrics

Model performance is evaluated on the held-out validation set of 596 images using two primary metrics. Mean Intersection over Union (mIoU) is the principal metric for both tasks, computed as the mean of per-class IoU scores across all valid classes. For the drivable area task, pixel-level accuracy is additionally reported. Given that early stopping was applied with patience of 3, model selection is assessed under two criteria: (1) the checkpoint at the final (early-stopped) epoch, representing the model as it would be deployed after

training completes, and (2) the checkpoint achieving the lowest validation loss (epoch 2 for all models), representing the best-generalising weights. Validation loss, training loss, and per-task mIoU are logged at every epoch for all three models to enable full analysis of training dynamics and overfitting behaviour.

To complement the quantitative evaluation, qualitative predictions from STL and MTL models are compared side-by-side against ground truth masks on representative validation images, providing visual evidence of differences in boundary sharpness, class confusion, and spatial coherence between the two training paradigms.

IV. Results and Discussion

This section presents the quantitative and qualitative results of the controlled empirical comparison between STL and MTL models. Results are reported at two checkpoints for each model: the best validation checkpoint (epoch 2, lowest validation loss for all three models) and the final early-stopped checkpoint (epoch 5). Reporting both checkpoints is necessary because the transfer effects differ meaningfully depending on which weights are compared, and using only one checkpoint would present an incomplete picture of model behaviour.

A. Quantitative Results

Table IV summarises the key performance metrics for all three model variants at both evaluation checkpoints. Bolded and shaded cells indicate the best-performing model for each task at each checkpoint.

TABLE IV
Best vs Final Checkpoint Performance Comparison

Model	Checkpoint	Seg mIoU	Drive mIoU	Drive Acc
STL-Seg	Best (E2)	0.2814	—	—
STL-Drive	Best (E2)	—	0.6936	93.57%
MTL	Best (E2)	**0.2945**	0.6508	92.90%
STL-Seg	Final (E5)	**0.3099**	—	—
STL-Drive	Final (E5)	—	**0.7189**	**94.25%**
MTL	Final (E5)	0.3074	0.6740	93.25%

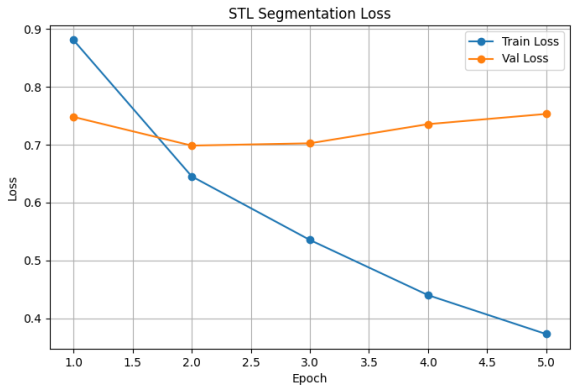

Fig. 2. STL-Seg training and validation loss across epochs

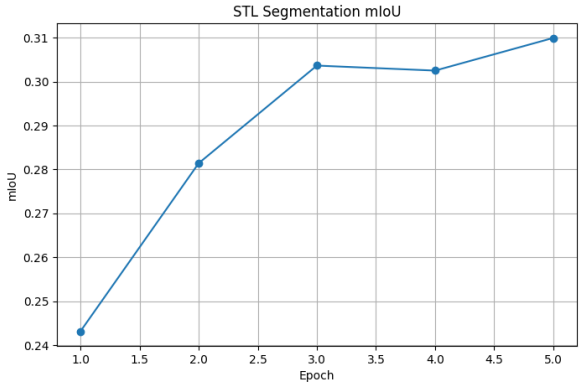

Fig. 3. STL-Seg validation mIoU across epochs

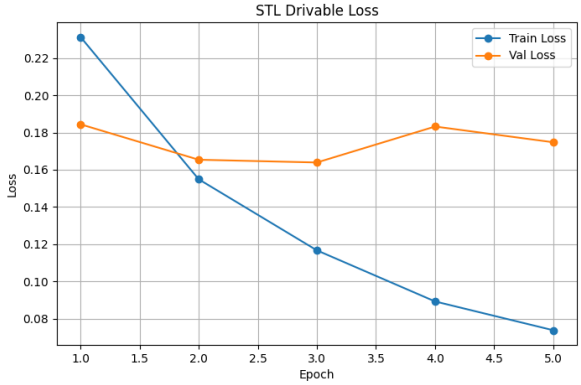

Fig. 4. STL-Drive training and validation loss across epochs

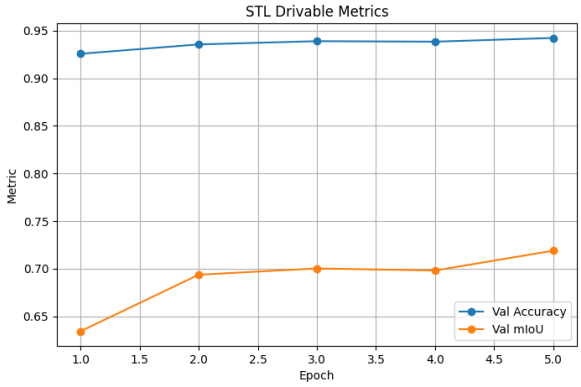

Fig. 5. STL-Drive validation accuracy and mIoU across epochs

At the best validation checkpoint (epoch 2), the MTL model achieves a segmentation mIoU of 0.2945, outperforming the STL-Seg baseline of 0.2814 by 1.31 percentage points. This indicates a positive transfer effect at early convergence, where the shared encoder provides beneficial contextual regularisation for the more complex segmentation task. However, for drivable area detection at the same checkpoint, the STL-Drive model achieves a substantially higher mIoU of 0.6936 compared to 0.6508 for MTL, a gap of 4.28 percentage points, indicating clear negative transfer. At the final early-stopped checkpoint (epoch 5), the transfer picture changes for segmentation. The STL-Seg model reaches a final mIoU of 0.3099, surpassing MTL's 0.3074 by 0.25 percentage points. The positive transfer observed at epoch 2 therefore does not persist to convergence, the STL segmentation model continues to improve through epochs 3 to 5 while the MTL segmentation head plateaus, ultimately overtaking it. For drivable area detection, the negative transfer effect is consistent and amplified at the final checkpoint, with STL-Drive (0.7189) exceeding MTL (0.6740) by 4.49 percentage points. These results establish two key findings. First, negative transfer is consistent and substantial for drivable area detection regardless of checkpoint, suggesting that competition from the segmentation task in the shared encoder fundamentally limits drivable area specialisation. Second, the apparent positive transfer for segmentation observed at the best-loss checkpoint is transient, it reflects an early regularisation benefit that disappears as the STL segmentation model continues to improve beyond the MTL model's plateau.

B. Transfer Effect Analysis

Table V summarizes the transfer effects at both checkpoints. The delta mIoU column reports the difference MTL to STL, where a positive value indicates positive transfer (MTL benefits from joint training) and a negative value indicates negative transfer (MTL is harmed by joint training).

TABLE V
TRANSFER EFFECTS FOR STL VS MTL ACROSS CHECKPOINTS

Task	Checkpoint	STL mIoU	MTL mIoU	Δ mIoU / Transfer
Segmentation	Best (E2)	0.2814	0.2945	**+0.0131 / Positive**
Segmentation	Final (E5)	0.3099	0.3074	-0.0025 / Negative
Drivable Area	Best (E2)	0.6936	0.6508	-0.0428 / Negative
Drivable Area	Final (E5)	0.7189	0.6740	-0.0449 / Negative

The most important observation from Table V is that the sign of the transfer effect for segmentation flips between checkpoints. At epoch 2 it is $+0.0131$ (positive), and at epoch 5 it is -0.0025 (negative). This checkpoint-dependence means that any claim of positive transfer for segmentation must be qualified: it is an early-training phenomenon only, and does not hold at the point where the STL model has fully converged. In contrast, the transfer effect for drivable area detection is negative at both checkpoints (-0.0428 and -0.0449), and of a substantially larger magnitude, indicating a robust and persistent negative transfer effect.

The asymmetry in transfer magnitude is informative. The negative transfer for drivable area (4.28–4.49 percentage points) is approximately 17 times larger in absolute terms than the transient positive transfer for segmentation (1.31 points at best). This asymmetry suggests that the shared encoder is dominated by segmentation gradient signals — consistent with segmentation having 19 output classes versus 3 for drivable area, resulting in a larger and more complex loss landscape that may overwhelm the drivable task's gradient contribution during joint optimisation.

C. Training Dynamics

Table VI presents the epoch-by-epoch validation loss and mIoU for all three models. The † symbol marks the epoch at which each model achieved its lowest validation loss.

TABLE VI
TRAINING DYNAMICS ACROSS EPOCHS

E	STL-S Loss	STL-S mIoU	STL-D Loss	STL-D mIoU	MTL Loss	MTL Seg mIoU	MTL Drv mIoU
1	0.748	0.243	0.184	0.634	0.967	0.268	0.629
2	**0.698**†	0.281	0.165	0.694	**0.859**†	0.295	0.651
3	0.702	0.304	**0.164**†	0.700	0.876	0.301	0.667
4	0.736	0.303	0.183	0.698	0.921	0.302	0.673
5	0.753	**0.310**	0.175	**0.719**	0.945	**0.307**	**0.674**

† indicates the epoch with the lowest validation loss for that model. Bold values indicate the highest mIoU.

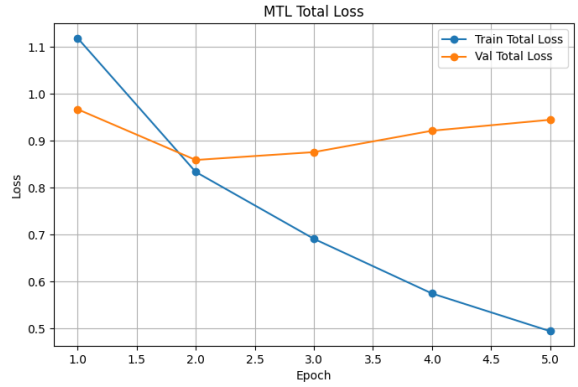

Fig. 6. MTL total training and validation loss across epochs

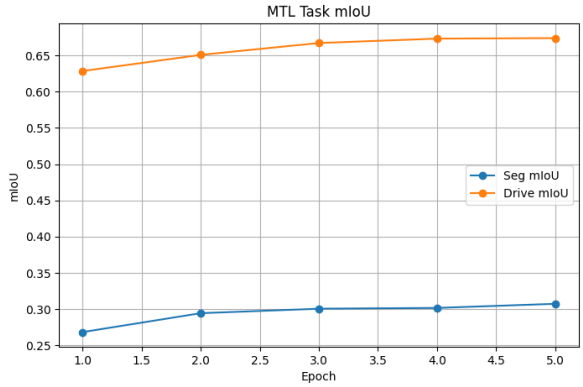

Fig. 7. MTL per-task validation mIoU across epochs

All three models achieve their lowest validation loss at epoch 2, after which validation loss rises while training loss

continues to decrease — a classic signature of overfitting in the shared representations. The MTL model shows the most pronounced divergence: its total validation loss rises from 0.859 at epoch 2 to 0.945 at epoch 5, an increase of 0.086, while training loss falls from 0.834 to 0.494 over the same period. This widening train-validation gap indicates that the jointly-trained encoder is fitting training-set patterns that do not generalise well, likely because the two tasks impose competing constraints on the shared feature space.

Despite the rising validation loss, per-task mIoU continues to improve beyond epoch 2 for all models. This apparent contradiction — worsening loss but improving mIoU — reflects the different sensitivity of the two metrics to calibration and class distribution. The mIoU metric averages over all classes and is less sensitive to small probability shifts across the full distribution, while cross-entropy loss penalises any miscalibration. The early-stopping criterion (patience=3 on validation loss) therefore triggers at epoch 5 for all models, which is conservative but appropriate: the mIoU gains from epochs 3 to 5 are marginal (less than 0.7 percentage points for any model), and the rising validation loss indicates that the improvements are not robust.

A notable difference in training dynamics between the STL and MTL models is the rate of mIoU improvement after epoch 2. The STL-Seg model improves from 0.281 at epoch 2 to 0.310 at epoch 5 — a gain of 2.9 percentage points over three epochs. The MTL segmentation head improves from 0.295 to 0.307 over the same period — a gain of only 1.2 percentage points. The STL-Drive model shows a similar pattern, improving by 2.53 percentage points (0.694 to 0.719) while the MTL drivable head improves by only 2.32 percentage points (0.651 to 0.674). This consistently lower post-epoch-2 improvement rate in MTL suggests that the shared encoder becomes a bottleneck for task-specific refinement once the joint representations have converged, limiting how much each decoder head can specialise beyond the shared feature level.

D. Qualitative Results

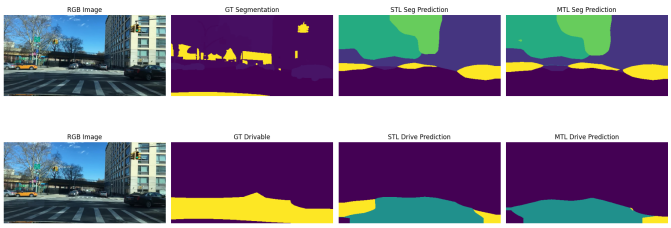

Fig. 8. Qualitative comparison: Urban Intersection Scene.

Figures 8, 9 and 10 present side-by-side qualitative comparisons of STL and MTL predictions against ground truth masks across three representative validation scenes, each showing both the semantic segmentation and drivable area predictions for the same input image.

For semantic segmentation, the STL and MTL predictions are broadly similar in terms of large-region class assignment

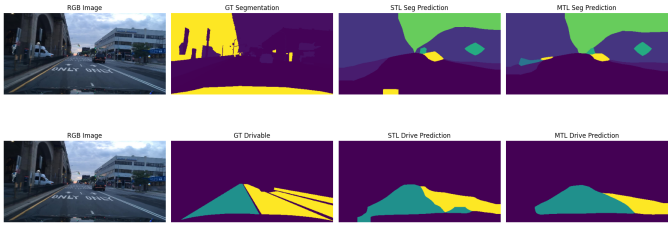

Fig. 9. Qualitative comparison: Dusk Urban Scene.

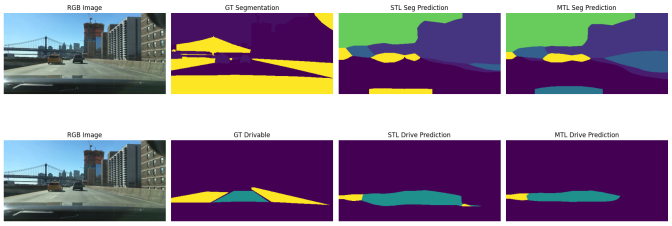

Fig. 10. Qualitative comparison: Highway Scene.

— both models correctly identify the sky, road surface, and building regions in most scenes. However, in scenes with multiple overlapping object classes (Fig. 9, the dusk scene with a vehicle ahead and visible lane markings), the MTL model shows slightly more coherent class boundaries in the mid-range of the image, consistent with the marginal mIoU advantage observed at the best checkpoint. At the final epoch, where STL-Seg numerically surpasses MTL-Seg, the differences are subtle and within the expected noise range of visual inspection.

For drivable area detection, the qualitative differences are more pronounced and consistently favour the STL model. In Fig. 8 (urban intersection scene), the STL-Drive prediction more closely matches the ground truth boundary between the directly drivable region (yellow) and the alternative drivable region (teal), whereas the MTL drivable head over-segments the alternative drivable area and blurs the boundary between classes. This pattern is repeated in Fig. 10 (highway scene), where the STL model correctly distinguishes the narrow directly drivable corridor from the broader alternative region, while the MTL prediction merges these into a larger, less precise region. These observations align with the quantitative finding of consistent negative transfer for drivable area detection and suggest that the loss of task-specific specialisation in the shared encoder manifests as reduced boundary precision and class discrimination for the simpler drivable task.

E. Discussion: Positive and Negative Transfer

The experimental findings reveal a nuanced picture of multi-task learning that differs from a simple positive or negative characterisation. The results are summarised conceptually in Table V and interpreted here in terms of the mechanisms that likely underlie the observed transfer effects. Drivable area detection experiences consistent, substantial negative transfer under MTL. The STL-Drive model outperforms the MTL drivable head by 4.28–4.49 mIoU percentage points depending

on checkpoint. The most plausible explanation is gradient domination: the segmentation task, with 19 output classes and a correspondingly large and complex loss surface, generates gradient signals that are larger in magnitude and richer in structure than those from the 3-class drivable task. When these gradients are summed with equal weighting in the shared encoder, the segmentation task effectively drives the majority of the encoder's parameter updates, leaving the drivable task with a shared representation that is optimised for multi-class boundary detection rather than for the coarser spatial discrimination required by drivable area prediction. This interpretation is consistent with the findings of Kang [4], who identified task gradient imbalance as a primary mechanism of negative transfer in MTL settings.

For semantic segmentation, the transfer effect is checkpoint-dependent and ultimately neutral. At the best validation checkpoint (epoch 2), MTL provides a 1.31 percentage point mIoU advantage over STL-Seg, suggesting that early in training the shared encoder provides useful cross-task regularisation — the drivable area task's simpler, spatially smoother supervision signal may help the shared encoder learn more stable low-level features that benefit segmentation. However, this advantage disappears by epoch 5, when the STL-Seg model surpasses MTL by 0.25 percentage points. The STL-Seg model continues to improve more rapidly after epoch 2 (2.9 pp gain) compared to the MTL segmentation head (1.2 pp gain), suggesting that the shared encoder becomes a constraint on task-specific refinement as training progresses. The net effect of multi-task training on segmentation is therefore approximately neutral, with a modest early benefit that does not persist to convergence.

These findings have practical implications for the design of multi-task learning systems in autonomous driving. When tasks differ substantially in class granularity and decision boundary complexity — as is the case with 19-class semantic segmentation and 3-class drivable area detection — equal-weight joint training may systematically disadvantage the simpler task. Adaptive loss weighting strategies, such as uncertainty-based weighting or gradient normalisation, that dynamically balance task contributions during training may be necessary to achieve consistent positive transfer across tasks of differing complexity. The results also suggest that the hard parameter sharing paradigm, in which the encoder is fully shared, may be too rigid for this particular task combination — architectures with partially shared or task-gated encoders may better accommodate the differing feature requirements of the two tasks.

V. Conclusion

This study presented a controlled empirical comparison between Single-Task Learning (STL) and Multi-Task Learning (MTL) for semantic segmentation and drivable area detection using the BDD100K dataset. Both approaches were implemented using a consistent ResNet-18 backbone with identical training hyperparameters, ensuring that any observed performance differences are attributable solely to the single-task versus multi-task training paradigm.

The experimental results reveal a nuanced and checkpoint-dependent picture of transfer effects. For drivable area detection, MTL produced consistent and substantial negative transfer across both evaluation checkpoints, with the STL-Drive model outperforming the MTL drivable head by 4.28–4.49 mIoU percentage points. The most plausible mechanism is gradient domination: the 19-class segmentation task generates larger and more complex gradient signals than the 3-class drivable task, causing the shared encoder to be optimised predominantly for segmentation-relevant features. For semantic segmentation, the transfer effect is checkpoint-dependent and ultimately neutral. MTL provided a marginal early advantage of 1.31 percentage points at epoch 2, suggesting a transient regularisation benefit from shared training. However, this advantage did not persist to convergence. By epoch 5, the STL-Seg model surpassed the MTL segmentation head, as the shared encoder became a bottleneck on task-specific refinement.

Training dynamics analysis revealed overfitting in the jointly-trained encoder, evidenced by a widening train-validation gap in MTL despite early stopping. Qualitative comparisons confirmed the quantitative findings: drivable area boundaries were consistently less precise under MTL, reflecting the loss of task-specific specialisation in the shared encoder.

Overall, these findings demonstrate that equal-weight hard parameter sharing is insufficient when tasks differ substantially in class granularity and loss complexity. Future work should explore adaptive loss weighting strategies such as uncertainty-based weighting or gradient normalisation, as well as partially shared or task-gated encoder architectures, to better accommodate tasks with differing feature requirements and reduce competition in the shared parameter space.

References

[1] K. Muhammad et al., "Vision-Based Semantic Segmentation in Scene Understanding for Autonomous Driving: Recent Achievements, Challenges, and Outlooks," in IEEE Transactions on Intelligent Transportation Systems, vol. 23, no. 12, pp. 22694-22715, Dec. 2022, doi: 10.1109/TITS.2022.3207665.

[2] D. Qiao and F. Zulkernine, "Drivable Area Detection Using Deep Learning Models for Autonomous Driving," 2021 IEEE International Conference on Big Data (Big Data), Orlando, FL, USA, 2021, pp. 5233-5238, doi: 10.1109/BigData52589.2021.9671392.

[3] Yaran Chen, Dongbin Zhao, Le Lv, Qichao Zhang, Multi-task learning for dangerous object detection in autonomous driving, Information Sciences, Volume 432, 2018, Pages 559-571, ISSN 0020-0255, https://doi.org/10.1016/j.ins.2017.08.035.

[4] Kang, Sungwoo. "Multi-Task Learning for Metal Alloy Property Prediction: An Empirical Study of Negative Transfer and Mitigation Strategies." (2025).

[5] Yu, F., Chen, H., Wang, X., Xian, W., Chen, Y., Liu, F., Madhavan, V., and Darrell, T. (2020). BDD100K: a diverse driving dataset for heterogeneous multitask learning. Berkley BAIR, 2633–2642. https://doi.org/10.1109/cvpr42600.2020.00271.

[6] Shelhamer, E., Long, J., and Darrell, T. (2016). Fully convolutional networks for semantic segmentation. IEEE Transactions on Pattern Analysis and Machine Intelligence, 39(4), 640–651. https://doi.org/10.1109/tpami.2016.2572683

[7] V. Badrinarayanan, A. Kendall and R. Cipolla, ”SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation,” in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 39, no. 12, pp. 2481-2495, 1 Dec. 2017, doi: 10.1109/TPAMI.2016.2644615.

[8] L. -C. Chen, G. Papandreou, I. Kokkinos, K. Murphy and A. L. Yuille, ”DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs,” in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 40, no. 4, pp. 834-848, 1 April 2018, doi: 10.1109/TPAMI.2017.2699184.

[9] Chen, L., Zhu, Y., Papandreou, G., Schroff, F., and Adam, H. (2018). Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation. In Lecture notes in computer science (pp. 833–851). https://doi.org/10.1007/978-3-030-01234-249.

[10] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M. Alvarez, and Ping Luo. 2021. SegFormer: simple and efficient design for semantic segmentation with transformers. In Proceedings of the 35th International Conference on Neural Information Processing Systems (NIPS ’21). Curran Associates Inc., Red Hook, NY, USA, Article 924, 12077–12090.

[11] Lee, D.-G. Fast Drivable Areas Estimation with Multi-Task Learning for Real-Time Autonomous Driving Assistant. Appl. Sci. 2021, 11, 10713. https://doi.org/10.3390/app112210713

[12] R. Caruana, ”Multitask Learning,” Mach. Learn., vol. 28, no. 1, pp. 41–75, 1997.

[13] M. Teichmann, M. Weber, M. Zöllner, R. Cipolla, and R. Urtasun, ”MultiNet: Real-time Joint Semantic Reasoning for Autonomous Driving,” in Proc. IEEE Intell. Veh. Symp., 2018.

[14] D. Wu, M.-W. Liao, W.-T. Zhang, X.-G. Wang, X. Bai, W.-Q. Cheng, and W.-Y. Liu, ”YOLOP: You Only Look Once for Panoptic Driving Perception,” Mach. Intell. Res., vol. 19, no. 6, pp. 550–562, 2022.

[15] D. Vu, B. Ngo, and H. Phan, ”HybridNets: End-to-End Perception Network,” arXiv:2203.09035, 2022.

[16] J. Zhan, J. Liu, Y. Wu, and C. Guo, ”Multi-Task Visual Perception for Object Detection and Semantic Segmentation in Intelligent Driving,” Remote Sens., vol. 16, no. 10, p. 1774, May 2024.

[17] J. Wang, Q. M. J. Wu, and N. Zhang, ”You Only Look At Once for Real-Time and Generic Multi-Task,” IEEE Trans. Veh. Technol., 2024, doi: 10.1109/TVT.2024.3394350.